



20-MAVZU · 42 TADAN · II BOB. JAVASCRIPT ASOSLARI

HTML eventlari va sahifaga interaktivlik. JavaScript mini loyiha

Loyihani rejalashtirish, "holat → chizish" usuli, forma tekshirish; To-Do List (localStorage bilan), mini test tizimi va interaktiv komponentlar — bosqichma-bosqich.

2 soat · amaliy

Bu darsda nimani o'rganasiz

- Loyihani boshlashdan oldin **rejalashtirishni**;
- Professional usul: ma'lumotni **massivda** saqlab, sahifani undan **chizish** (render);
- Formani JavaScript bilan tekshirish (validatsiya);
- To'liq **To-Do List** ilovasini (11-loyiha);
- **Mini test tizimini** (12-loyiha) yaratishni.

Bu dars — II bobning yakuni: o'zgaruvchilar, shartlar, sikllar, funksiyalar, massivlar, DOM va hodisalar bitta loyihada birlashadi.

Loyihani qanday boshlash kerak?

Yangi boshlovchilarning eng katta xatosi — darhol kod yozishga tushish. Tajribali dasturchi avval o'ylaydi:

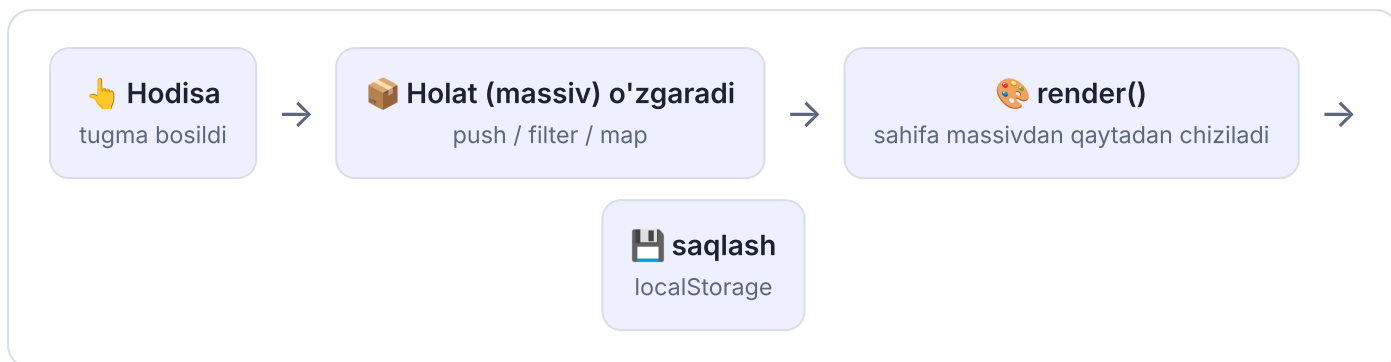
- 1 **Nima qilishi kerak?** Funksiyalar ro'yxatini yozing: "vazifa qo'shish", "bajarildi deb belgilash", "o'chirish"...
- 2 **Qanday ko'rinadi?** Qog'ozga chizib oling: input, tugma, ro'yxat.
- 3 **Qanday ma'lumot saqlanadi?** Har bir vazifa — obyekt: `{ id, matn, bajarildi }`; hammasi — massiv.

4 **Qaysi hodisalar bor?** Forma yuborildi → qo'shish; checkbox bosildi → belgilash; 🗑️ bosildi → o'chirish.

5 **Kichik qadamlar bilan yozing** va har qadamdan keyin tekshiring.

"Holat → Chizish" usuli

Ro'yxatga element qo'shganda ham, o'chirganda ham DOM'ni qo'lda "yamash" chalkashlikka olib keladi. Professional usul:



O'xshatish: doska va daftar

Daftar — **holat** (haqiqiy ma'lumot). Doska — **ekran**. O'qituvchi doskaga to'g'ridan-to'g'ri yozib-o'chirib o'tirmaydi: avval daftarni yangilaydi, keyin doskani artib, daftardagini qaytadan yozadi. Shunda doska va daftar hech qachon farq qilmaydi.

Aynan shu g'oya React kabi zamonaviy kutubxonalarning asosida yotadi.

1-loyiha: To-Do List (11-loyiha)

1-qadam: HTML va CSS

todo/index.html

```
<!DOCTYPE html>
<html lang="uz">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Vazifalarim</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <main class="ilova">
    <h1>📝 Vazifalarim</h1>

    <form id="forma">
      <input id="kiritish" placeholder="Yangi vazifa..." maxlength="80"
autocomplete="off">
      <button>Qo'shish</button>
    </form>
    <p id="xato" class="xato"></p>

    <div class="filtrlar">
      <button data-filtr="hammasi" class="faol">Hammasi</button>
      <button data-filtr="faol">Faol</button>
      <button data-filtr="bajarilgan">Bajarilgan</button>
    </div>

    <ul id="royxat"></ul>

    <footer class="pastki">
      <span id="hisob"></span>
      <button id="tozalash">Bajarilganlarni o'chirish</button>
    </footer>
  </main>
  <script src="app.js"></script>
</body>
</html>
```

2-qadam: JavaScript — bosqichma-bosqich

todo/app.js

```
// ===== 1. HOLAT (ma'lumotlar) =====
let vazifalar = JSON.parse(localStorage.getItem("vazifalar")) || [];
let filtr = "hammasi";

// ===== 2. ELEMENTLAR =====
const forma = document.querySelector("#forma");
const kiritish = document.querySelector("#kiritish");
const royxat = document.querySelector("#royxat");
const xato = document.querySelector("#xato");
const hisob = document.querySelector("#hisob");

// ===== 3. AMALLAR (holatni o'zgartiradi) =====
function qoshish(matn) {
  vazifalar.push({ id: Date.now(), matn: matn, bajarildi: false });
  yangilash();
}

function belgilash(id) {
  vazifalar = vazifalar.map(v => v.id === id ? { ...v, bajarildi: !v.bajarildi } : v);
  yangilash();
}

function ochirish(id) {
  vazifalar = vazifalar.filter(v => v.id !== id);
  yangilash();
}

function yangilash() {
  localStorage.setItem("vazifalar", JSON.stringify(vazifalar)); // saqlash
  render(); // chizish
}

// ===== 4. CHIZISH (holatdan sahifa) =====
function render() {
  royxat.innerHTML = ""; // eski ro'yxatni tozalash

  const korinadigan = vazifalar.filter(v => {
    if (filtr === "faol") return !v.bajarildi;
  });
}
```

```

    if (filtr === "bajarilgan") return v.bajarildi;
    return true;
  });

  if (korinadigan.length === 0) {
    royxat.innerHTML = '<li class="bosh">Bu yerda hozircha hech narsa yo\'q 🤖';
  }

  korinadigan.forEach(v => {
    const li = document.createElement("li");
    if (v.bajarildi) li.classList.add("bajarildi");

    const checkbox = document.createElement("input");
    checkbox.type = "checkbox";
    checkbox.checked = v.bajarildi;
    checkbox.addEventListener("change", () => belgilash(v.id));

    const span = document.createElement("span");
    span.textContent = v.matn; // ✅ textContent – xavfsiz

    const ochirishTugma = document.createElement("button");
    ochirishTugma.textContent = "🗑️";
    ochirishTugma.title = "O'chirish";
    ochirishTugma.addEventListener("click", () => ochirish(v.id));

    li.append(checkbox, span, ochirishTugma);
    royxat.append(li);
  });

  const qolgan = vazifalar.filter(v => !v.bajarildi).length;
  hisob.textContent = `${qolgan} ta vazifa qoldi`;
}

// ===== 5. HODISALAR =====
forma.addEventListener("submit", (e) => {
  e.preventDefault();
  const matn = kiritish.value.trim();

  // Validatsiya
  if (matn === "") {
    xato.textContent = "⚠️ Vazifa matnini kiriting";
    return;
  }
}

```

```

    if (vazifalar.some(v => v.matn.toLowerCase() === matn.toLowerCase())) {
        xato.textContent = "⚠ Bunday vazifa allaqachon bor";
        return;
    }

    xato.textContent = "";
    qoshish(matn);
    kiritish.value = "";
    kiritish.focus();
});

document.querySelectorAll("[data-filtr]").forEach(tugma => {
    tugma.addEventListener("click", () => {
        filtr = tugma.dataset.filtr; // data-filtr atributining
        qiymati
        document.querySelectorAll("[data-filtr]").forEach(t =>
        t.classList.remove("faol"));
        tugma.classList.add("faol");
        render();
    });
});

document.querySelector("#tozalash").addEventListener("click", () => {
    vazifalar = vazifalar.filter(v => !v.bajarildi);
    yangilash();
});

// ===== 6. BOSH LASH =====
render();

```

3-qadam: dizayn

todo/style.css

```
* { box-sizing: border-box; }
body { font-family: "Segoe UI", sans-serif; background: #eef2ff; margin: 0;
padding: 40px 16px; }
.ilova { max-width: 480px; margin: 0 auto; background: white; padding: 28px;
border-radius: 18px;
    box-shadow: 0 15px 35px rgba(79,70,229,.12); }
h1 { margin-top: 0; text-align: center; }
#forma { display: flex; gap: 8px; }
#forma input { flex: 1; padding: 12px; border: 2px solid #e5e7eb; border-
radius: 10px; font-size: 16px; }
#forma input:focus { border-color: #4f46e5; outline: none; }
#forma button { padding: 0 18px; border: none; border-radius: 10px; background:
#4f46e5; color: white; font-weight: 600; cursor: pointer; }
.xato { color: #dc2626; min-height: 1.2em; margin: 6px 0; font-size: 14px; }
.filtrlar { display: flex; gap: 6px; margin-bottom: 10px; }
.filtrlar button { border: 1px solid #e5e7eb; background: white; padding: 6px
12px; border-radius: 999px; cursor: pointer; }
.filtrlar button.faol { background: #4f46e5; color: white; border-color:
#4f46e5; }
#royxat { list-style: none; padding: 0; margin: 0; }
#royxat li { display: flex; align-items: center; gap: 10px; padding: 12px 4px;
border-bottom: 1px solid #f3f4f6; }
#royxat li span { flex: 1; }
#royxat li.bajarildi span { text-decoration: line-through; color: #9ca3af; }
#royxat li button { border: none; background: none; cursor: pointer; font-size:
18px; opacity: .4; transition: opacity .2s; }
#royxat li:hover button { opacity: 1; }
#royxat li.bosh { justify-content: center; color: #9ca3af; }
.pastki { display: flex; justify-content: space-between; margin-top: 14px;
font-size: 14px; color: #6b7280; }
.pastki button { border: none; background: none; color: #6b7280; text-
decoration: underline; cursor: pointer; }
```

Uchala faylni bitta papkada yarating va Live Server bilan oching. Vazifalar qo'shing, sahifani yangilang — ular **saqlanib qoladi** (localStorage).



Tushunish uchun tajriba

DevTools → **Application** → **Local Storage** ni oching. Vazifa qo'shganingizda `vazifalar` kaliti ostidagi JSON qanday o'zgarishini kuzating.

2-loyiha: Mini test tizimi (12-loyiha)

Savollar **ma'lumot** sifatida massivda saqlanadi — yangi savol qo'shish uchun kodni o'zgartirish shart emas, faqat massivga qo'shish kifoya.

```

<!DOCTYPE html>
<html lang="uz">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Mini test</title>
<style>
  body { font-family: "Segoe UI", sans-serif; background: #f1f5f9; display:
grid; place-items: center; min-height: 100vh; margin: 0; }
  .test { background: white; width: min(480px, 92vw); padding: 28px; border-
radius: 18px; box-shadow: 0 10px 30px rgba(0,0,0,.08); }
  .progress { height: 8px; background: #e5e7eb; border-radius: 4px; overflow:
hidden; margin-bottom: 18px; }
  .progress div { height: 100%; background: #4f46e5; width: 0; transition:
width .4s; }
  .raqam { color: #6b7280; font-size: 14px; }
  h2 { margin: 6px 0 16px; font-size: 1.25rem; }
  .variant { display: block; width: 100%; text-align: left; padding: 12px 14px;
margin: 8px 0; border: 2px solid #e5e7eb;
    border-radius: 10px; background: white; font-size: 16px; cursor:
pointer; transition: .2s; }
  .variant:hover:not(:disabled) { border-color: #4f46e5; background: #eef2ff; }
  .variant.togri { border-color: #22c55e; background: #dcfce7; }
  .variant.notogri { border-color: #ef4444; background: #fee2e2; }
  #keyingi { margin-top: 14px; padding: 12px 22px; border: none; border-radius:
10px; background: #4f46e5; color: white; font-size: 16px; cursor: pointer; }
  .yashirin { display: none; }
  .natija { text-align: center; }
  .natija .ball { font-size: 3rem; font-weight: 800; color: #4f46e5; }
</style>
</head>
<body>
<div class="test">
  <div class="progress"><div id="chiziq"></div></div>
  <div id="savolBloki">
    <div class="raqam" id="raqam"></div>
    <h2 id="savol"></h2>
    <div id="variantlar"></div>
    <button id="keyingi" class="yashirin">Keyingi →</button>
  </div>
  <div id="natijaBloki" class="natija yashirin"></div>
</div>

```

```

<script>
  const SAVOLLAR = [
    { savol: "Sahifa tuzilishini yaratuvchi til qaysi?", variantlar: ["CSS",
"HTML", "JavaScript", "Python"], togri: 1 },
    { savol: "Doim ishlatilishi kerak bo'lgan taqqoslash operatori?",
variantlar: ["=", "==", "===", "⇒"], togri: 2 },
    { savol: "Massiv oxiriga element qo'shuvchi metod?", variantlar: ["pop()",
"push()", "shift()", "add()"], togri: 1 },
    { savol: "Elementni CSS selektori bilan topuvchi metod?", variantlar:
["querySelector", "findElement", "getCSS", "select"], togri: 0 },
    { savol: "17 % 5 nechaga teng?", variantlar: ["3", "3.4", "2", "12"],
togri: 2 },
  ];

  let joriy = 0;
  let ball = 0;

  const raqamEl = document.querySelector("#raqam");
  const savolEl = document.querySelector("#savol");
  const variantlarEl = document.querySelector("#variantlar");
  const keyingiEl = document.querySelector("#keyingi");
  const chiziqEl = document.querySelector("#chiziq");

  function savolniKorsat() {
    const s = SAVOLLAR[joriy];
    raqamEl.textContent = `Savol ${joriy + 1} / ${SAVOLLAR.length}`;
    savolEl.textContent = s.savol;
    chiziqEl.style.width = `${(joriy / SAVOLLAR.length) * 100}%`;
    keyingiEl.classList.add("yashirin");
    variantlarEl.innerHTML = "";

    s.variantlar.forEach((matn, i) => {
      const tugma = document.createElement("button");
      tugma.className = "variant";
      tugma.textContent = matn;
      tugma.addEventListener("click", () => javobBer(i, tugma));
      variantlarEl.append(tugma);
    });
  }

  function javobBer(tanlangan, tugma) {
    const togri = SAVOLLAR[joriy].togri;
    const hammasi = variantlarEl.querySelectorAll(".variant");
  }

```

```

        hammasi.forEach(t => t.disabled = true); // qayta bosib
        bo'lmasin

        if (tanlangan === togri) {
            ball++;
            tugma.classList.add("togri");
        } else {
            tugma.classList.add("notogri");
            hammasi[togri].classList.add("togri"); // to'g'ri javobni
        }
    }
    ko'rsatamiz
    keyingiEl.classList.remove("yashirin");
}

keyingiEl.addEventListener("click", () => {
    joriy++;
    if (joriy < SAVOLLAR.length) savolniKorsat();
    else natijaniKorsat();
});

function natijaniKorsat() {
    chiziqEl.style.width = "100%";
    document.querySelector("#savolBlok").classList.add("yashirin");
    const foiz = Math.round((ball / SAVOLLAR.length) * 100);
    const izoh = foiz ≥ 86 ? "A'lo! 🏆" : foiz ≥ 60 ? "Yaxshi! 👍" : "Yana
    mashq qiling 📖";
    const blok = document.querySelector("#natijaBlok");
    blok.classList.remove("yashirin");
    blok.innerHTML = `<p>Natijangiz</p><div
    class="ball">${ball}/${SAVOLLAR.length}</div>
    <p>${foiz}% – ${izoh}</p><button id="qayta">Qaytadan
    boshlash</button>`;
    document.querySelector("#qayta").addEventListener("click", () => {
        joriy = 0; ball = 0;
        blok.classList.add("yashirin");
        document.querySelector("#savolBlok").classList.remove("yashirin");
        savolniKorsat();
    });
}

savolniKorsat();
</script>
</body>
</html>

```

Kengaytirish g'oyalari:

- 1 O'zingiz 10 ta savol yozing (HTML, CSS, JS bo'yicha).
- 2 Savollar va variantlar har safar **aralashtirilib** chiqsin (Maslahat: `arr.sort(() => Math.random() - 0.5)`; variantlarni aralashtirganda to'g'ri javob indeksini ham yangilashni unutmang!).
- 3 Har bir savolga 20 soniyalik taymer qo'shing (`setInterval`). Vaqt tugasa — javob noto'g'ri hisoblanadi.
- 4 Eng yaxshi natijani `localStorage` da saqlang va "Rekord: 9/10" deb ko'rsating.

Forma validatsiyasi: real vaqtda tekshirish

HTML

```
<style>
  body { font-family: sans-serif; padding: 20px; }
  .maydon { margin-bottom: 14px; }
  input { padding: 10px; border: 2px solid #d1d5db; border-radius: 8px; width:
260px; }
  .maydon.xato input { border-color: #ef4444; }
  .maydon.ok input { border-color: #22c55e; }
  small { display: block; color: #ef4444; min-height: 1em; }
</style>
<form id="royxatdan">
  <div class="maydon"><input name="email" placeholder="Email"><small></small>
</div>
  <div class="maydon"><input name="parol" type="password" placeholder="Parol">
<small></small></div>
  <button>Ro'yxatdan o'tish</button>
</form>
<script>
  const qoidalar = {
    email: v => /^\\S+@\\S+\\.\\S+$/\\.test(v) ? "" : "Email noto'g'ri",
    parol: v => v.length < 8 ? "Kamida 8 belgi" : !\\/d/.test(v) ? "Kamida bitta
raqam bo'lsin" : "",
  };
  const forma = document.querySelector("#royxatdan");

  function tekshir(input) {
    const xato = qoidalar[input.name](input.value);
    const maydon = input.closest(".maydon");
    maydon.querySelector("small").textContent = xato;
    maydon.classList.toggle("xato", xato !== "");
    maydon.classList.toggle("ok", xato === "");
    return xato === "";
  }

  forma.addEventListener("input", e => tekshir(e.target));
  forma.addEventListener("submit", e => {
    e.preventDefault();
    const hammasiOk = [...forma.elements].filter(el =>
el.name).map(tekshir).every(Boolean);
    if (hammasiOk) alert("✅ Muvaffaqiyatli ro'yxatdan o'tdingiz!");
  });
</script>
```

```
});  
</script>
```

`/^\s+@|\s+\.\s+$/` — **muntazam ifoda (regex)**: "bo'sh joydiz belgilar, @, belgilar, nuqta, belgilar".

Regex'lar matnni naqsh bo'yicha tekshirish uchun ishlatiladi.

Ko'p uchraydigan xatolar

- ✗ Holatni o'zgartirib, `render()` ni chaqirishni unutish — massiv o'zgardi, lekin ekran eski holicha.
- ✗ `localStorage` ga massivni `JSON.stringify` siz saqlash — `"[object Object]"` saqlanib qoladi.
- ✗ `render()` da ro'yxatni tozalamaslik (`innerHTML = ""`) — har safar elementlar ikki baravar ko'payadi.
- ✗ Foydalanuvchi matnini `innerHTML` bilan qo'shish — XSS xavfi. `textContent` !

O'zingizni tekshiring

? "Holat → Chizish" usulida tugma bosilganda birinchi nima o'zgaradi?

- A. Sahifadagi HTML
- B. CSS fayl
- C. Ma'lumotlar massivi (holat)
- D. `localStorage`

? **Massivni localStorage'da saqlash uchun avval nima qilinadi?**

- A. `JSON.stringify()` bilan matnga aylantiriladi
- B. `toString()` chaqiriladi
- C. Hech narsa, to'g'ridan-to'g'ri saqlanadi
- D. `Number()` bilan songa aylantiriladi

? **`<button data-filtr="faol">` dagi qiymatni JS'da qanday o'qiysiz?**

- A. `tugma.filtr`
- B. `tugma.data`
- C. `tugma.value`
- D. `tugma.dataset.filtr`

? **Vazifani o'chirishning eng qulay usuli (id bo'yicha)?**

- A. `vazifalar.pop(id)`
- B. `vazifalar = vazifalar.filter(v => v.id !== id)`
- C. `delete vazifalar[id]`
- D. `vazifalar.remove(id)`



Uyga vazifa (bob yakunlovchi loyiha)

Quyidagilardan **birini** tanlab, to'liq ilova yarating va GitHub + Netlify'ga joylang:

1. **Xarajatlar hisobchisi:** xarajat nomi, summasi va toifasi qo'shiladi; jami summa va toifalar bo'yicha yig'indi ko'rsatiladi; localStorage.
2. **Lug'at o'yini:** inglizcha so'z ko'rsatiladi, foydalanuvchi tarjimasini yozadi; ball va "qiyin so'zlar" ro'yxati.

3. **Kutubxona katalogi:** kitoblar kartochkalari, qidiruv maydoni (real vaqtda filtr), janr bo'yicha filtr, "o'qildi" belgisi.



II bob yakunlandi!

- Loyiha: reja → dizayn → ma'lumot tuzilmasi → hodisalar → kichik qadamlar.
- "Holat → Chizish": hodisa holatni o'zgartiradi, `render()` sahifani holatdan chizadi, `localStorage` saqlaydi.
- Siz endi interaktiv veb-illovalar yarata olasiz. Keyingi bobda **server tomoniga** o'tamiz — Python va ma'lumotlar bazalari!

☐ Darsni o'qib bo'ldim

✓ Test javoblari

1. "Holat → Chizish" usulida tugma bosilganda birinchi nima o'zgaradi? — **C) Ma'lumotlar massivi (holat)**
2. Massivni localStorage'da saqlash uchun avval nima qilinadi? — **A) JSON.stringify() bilan matnga aylantiriladi**
3. <button data-filtr="faol"> dagi qiymatni JS'da qanday o'qiysiz? — **D) tugma.dataset.filtr**
4. Vazifani o'chirishning eng qulay usuli (id bo'yicha)? — **B) vazifalar = vazifalar.filter(v ⇒ v.id !== id)**